



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Λειτουργικά Συστήματα
Εργασία 1(2025-2026)

Ονοματεπώνυμο : Τσάκωνας Βασίλης

Αριθμός Μητρώου : 1115202200245

Περιεχόμενα

1	Εισαγωγή	1
2	Δομή αρχείων	1
3	Σχεδιαστικές αρχές	2
3.1	Δομές συγχρονισμού	2
3.2	Αρχικοποίηση/καταστροφή δομών	2
3.3	Πρωτόκολλο ταυτόχρονης αποστολής και λήψης μηνυμάτων	2
3.4	Πρωτόκολλο τερματισμού	3
4	Παραδείγματα	4

1 Εισαγωγή

Σκοπός της παρούσας εργασίας είναι η ταυτόχρονη επικοινωνία μεταξύ διεργασιών. Οι διεργασίες μπορούν να στείλουν και να λάβουν μηνύματα ταυτόχρονα. Μπορούν να υπάρχουν ταυτόχρονα αρκετοί διάλογοι οπότε οι διεργασίες έχουν πρόσβαση μόνο στα μηνύματα του διαλόγου τους. Όταν ληφθεί από κάποια διεργασία η εντολή TERMINATE τότε το μήνυμα στέλνεται και στους υπόλοιπους συμμετέχοντες του διαλόγου και αφού διαβαστεί από όλους τότε τερματίζουν όλοι. Άρα ακόμη και ένας να στείλει το μήνυμα "TERMINATE" ο διάλογος τερματίζει. Αν ο διάλογος που τερματίζει είναι και ο τελευταίος τότε αναλαμβάνει να "καθαρίσει" τις δομές συγχρονισμού όλης της εφαρμογής.

2 Δομή αρχείων

Στον φάκελο OS1 υπάρχουν 2 φάκελοι (incl, src). Στον incl βρίσκονται τα .h αρχεία και στον src τα .c. Ο κώδικας έχει "σπάσει" σε επιμέρους αρχεία. Υπάρχει ένα αρχείο(shared_mem.c) που περιλαμβάνει τις συναρτήσεις που αρχικοποιούν και καταστρέφουν τη κοινόχρηστη μνήμη. Ένα άλλο αρχείο(threads.c) που αφορά τις συναρτήσεις των 2 νημάτων όπου το 1 στέλνει μηνύματα και το άλλο λαμβάνει. Ακόμη ένα (functions_dial.c) αφορούν συναρτήσεις αρχικοποίησης ή καταστροφής του εκάστοτε διαλόγου. Τέλος, το main.c αποτελεί το σημείο εκκίνησης της εφαρμογής και είναι υπεύθυνο για την αρχικοποίηση των δομών, τη δημιουργία των νημάτων και τον ορθό τερματισμό της διεργασίας και των πόρων που αυτή χρησιμοποιεί.

3 Σχεδιαστικές αρχές

read thread : stdin-> shared memory's buffer (αποστολή μηνύματος)

write thread: shared memory's buffer -> stdout (λήψη μηνύματος)

3.1 Δομές συγχρονισμού

Η ανταλλαγή μηνυμάτων γίνεται μέσω της διαμοιραζόμενης μνήμης. Η διαμοιραζόμενη μνήμη υλοποιείται μέσω της δομής (shared_mem) που αποτελείται από τους διαλόγους, έναν σηματοφόρο που προστατεύει τη δομή για αποφυγή race-condition και μια μεταβλητή (proc) που χρησιμοποιείται ως ο συνολικός αριθμός ενεργών συμμετεχόντων σε όλους τους διαλόγους μαζί.

Η δομή του διαλόγου περιέχει ένα flag για το αν είναι ο διάλογος ενεργός, το id του διαλόγου, τον αριθμό των συμμετεχόντων και το pid τους. Για την αποφυγή και ομαλή λειτουργία των διαλόγων υπάρχουν 3 σηματοφόροι. Ο σηματοφόρος dial_mutex γίνεται post() και wait() για να εξασφαλιστεί πως μόνο μια διεργασία έχει πρόσβαση στον διάλογο κατά τη διάρκεια του critical section. Ο σηματοφόρος can_send_mess δείχνει ότι το μήνυμα που είναι στο κοινόχρηστο buffer έχει διαβαστεί από όλους και όταν γίνει post() μπορεί τότε ένα read thread να γράψει ένα νέο μήνυμα στο buffer (αρχικοποίηση με 1). Τέλος, το κάθε write thread έχει το δικό του σηματοφόρο(can_be_read) που χρησιμοποιείται για να εξασφαλιστεί πως το κάθε write thread διάβασε το μήνυμα. Ο λόγος που δε χρησιμοποιήθηκε ένας κοινός σηματοφόρος για όλες τις διεργασίες είναι πως ο μηχανισμός του σηματοφόρου δεν εγγυάται πως η κάθε διεργασία θα εκτελεστεί μόνο μια φορά. Οπότε στη περίπτωση που έχουμε 3 διεργασίες (A, B, Γ) υπάρχει περίπτωση η διεργασία A να εκτελεστεί 2 φορές στο critical section, η B να εισέλθει 1 φορά και η Γ να μη μπορεί να εισέλθει. Αυτό θα συνέβαινε καθώς ο σηματοφόρος θα αρχικοποιούνταν με τιμή 3 όταν θα ήταν να διαβάσει η διεργασία 3 η τιμή θα ήταν 0 και άρα θα μπλόκαρε. Επομένως, τώρα η κάθε διεργασία έχει τον δικό της σηματοφόρο που αρχικοποιείται με 0. Όταν κάποιος στείλει ένα μήνυμα τότε το read thread της συγκεκριμένης διεργασίας κάνει post() σε όλους τους σηματοφόρους can_be_read των write threads του εκάστοτε διαλόγου και όταν το write thread διαβάσει το μήνυμα καλεί την wait() στον ατομικό του σηματοφόρο.

Επιπλέον, οι διάλογοι περιέχουν τη δομή ενός μηνύματος(Message) στον οποίο αποθηκεύεται το μήνυμα που μοιράζεται στις διεργασίες. Αυτή η δομή αποτελείται από τον buffer, το pid του αποστολέα και τον αριθμό των διεργασιών που έχουν διαβάσει το μήνυμα. Η εφαρμογή βασίζεται στο γεγονός ότι στον χώρο της διαμοιραζόμενης μνήμης υπάρχει μόνο ένα μήνυμα(buffer) που αποθηκεύεται. Παρ' όλα αυτά εξασφαλίζεται πως κανένα μήνυμα δε θα χαθεί ακόμα και αν 2 read threads θέλουν να στείλουν ένα μήνυμα την ίδια ακριβώς χρονική στιγμή.

Τέλος για να περάσουμε τα ορίσματα στα νήματα χρησιμοποιούμε τη δομή t_args που περιέχει το shared_mem, ένα pipe(δες παρακάτω) και το index στον πίνακα των pids στη δομή του διαλόγου.

3.2 Αρχικοποίηση/καταστροφή δομών

Η αρχικοποίηση όλων των σηματοφόρων της κοινόχρηστης μνήμης αλλά και όλων των διαλόγων γίνονται από τη πρώτη διεργασία που χρησιμοποιεί το shared_mem. Έτσι με την ίδια λογική η τελευταία διεργασία της εφαρμογής καταστρέφει αυτές τις δομές συγχρονισμού χωρίς να υπάρχουν leaks.

3.3 Πρωτόκολλο ταυτόχρονης αποστολής και λήψης μηνυμάτων

Για να μπορεί η εκάστοτε διεργασία να μπορεί να στείλει αλλά και να λάβει μηνύματα ταυτόχρονα χρησιμοποιούνται 2 threads. Ένα για τη λήψη μηνυμάτων (write thread) και ένα για την αποστολή(read thread). Το read thread διαβάζει από την είσοδο του χρήστη το μήνυμα που πρέπει να σταλεί. Στη συνέχεια κάνει wait() τον σηματοφόρο can_send_mess, που υποδεικνύει πως μπορεί να γράψει στο buffer καθώς όλες οι διεργασίες έχουν λάβει το προηγούμενο μήνυμα, και το dial_mutex που λειτουργεί ως mutex για να προστατεύσει τον διάλογο. Στη συνέχεια αντιγράφει το μήνυμα από την είσοδο στο buffer κάνει post() όλους τους σηματοφόρους can_read

για να ξυπνήσουν τα write threads και στο τέλος κάνει `post( )` και το `dial_mutex`.

Το κάθε write thread κάνει `wait( )` τον σηματοφόρο `can_read` που του αναλογεί και το `dial_mutex`. Στη συνέχεια χωρίζεται σε 2 περιπτώσεις. Αν είναι η διεργασία που έστειλε το μήνυμα δεν το εκτυπώνει καθώς δεν έχει νόημα(αφού αυτή το έστειλε). Οποιαδήποτε άλλη διεργασία λαμβάνει το μήνυμα, το εκτυπώνει. Επιπλέον, ελέγχεται αν το μήνυμα που λήφθηκε είναι `TERMINATE` ή αν είναι η τελευταία διεργασία που έλαβε το μήνυμα οπότε πρέπει να κάνει `post( )` τον σηματοφόρο `can_send_mess`.

Παρόλο που στον διάλογο μπορεί να μεταφερθεί μόνο ένα μήνυμα τη φορά με τη χρήση της `fgets()` και των σηματοφόρων εξασφαλίζουμε πως όσα μηνύματα και να έρθουν ταυτόχρονα εφόσον χωράνε στον buffer θα αποθηκευτούν, θα σταλθούν και κανένα δε θα χαθεί. Στην περίπτωση όπου μόνο μια διεργασία θέλει να στείλει την ίδια χρονική στιγμή πολλά μηνύματα τότε αυτά στέλνονται με τη σειρά που γράφτηκαν σε πραγματικό χρόνο στο terminal. Σε άλλη περίπτωση όμως όπου πολλές διεργασίες θέλουν να στείλουν ένα μήνυμα την ίδια χρονική στιγμή τότε το λειτουργικό, μέσω του scheduler, αποφασίζει τη σειρά εκτέλεσης των διεργασιών και κατ' επέκταση τη σειρά αποστολής των μηνυμάτων.

Επομένως η συνολική διαδρομή στο πρωτόκολλο ταυτόχρονης αποστολής και λήψης μηνυμάτων είναι:

1. λήψη μηνύματος από terminal
2. το read thread εξασφαλίζει πως μπορεί να στείλει νέο μήνυμα με τον σηματοφόρο `can_send_mess` (`wait`) και το αντιγράφει στο buffer
3. το read thread ξυπνάει τα write threads μέσω του σηματοφόρου `can_read` (`post`)
4. το write thread ενημερώνεται πως ήρθε μήνυμα (`wait`)
5. το write thread λαμβάνει από το buffer το μήνυμα και το εκτυπώνει
6. η τελευταία διεργασία που θα λάβει το μήνυμα ενημερώνει ότι τώρα οποιοδήποτε read thread μπορεί να στείλει το επόμενο μήνυμα μέσω του σηματοφόρου `can_send_mess(post)` εκτός αν το μήνυμα είναι `TERMINATE` που ξεκινάει το πρωτόκολλο τερματισμού

3.4 Πρωτόκολλο τερματισμού

Όταν μια διεργασία στείλει το μήνυμα `TERMINATE` τότε όλες πρέπει να τερματίσουν. Στην υλοποίηση αυτή ο έλεγχος γίνεται στο write thread. Στην περίπτωση όπου το μήνυμα είναι `terminate` έχει δημιουργηθεί ένα pipe όπου το read και το write thread επικοινωνούν μεταξύ τους. Το write στέλνει ένα bit(-1) και όταν το read το διαβάσει αυτό τερματίζει. Επιπλέον για να μπορούμε να παίρνουμε είσοδο στο read thread και από το stdin και από το `pipe[0]` κάνουμε χρήση της συνάρτησης `poll( )`. Αυτή η συνάρτηση περιμένει είσοδο και από τις 2 πηγές αποφεύγοντας όμως το busy-waiting. Έτσι μπορεί να διαβάξει και από το `pipe[0]` για να δει το `TERMINATE` και από την είσοδο `stdin` για να στείλει μηνύματα από τον χρήστη.

Ο λόγος που έγινε χρήση του pipe για τον τερματισμό είναι πως η συνάρτηση `fgets()` στο read thread είναι blocking. Επομένως το `read_thread` δε μπορεί να ξεμπλοκαριστεί χωρίς να δώσει κάποιος είσοδο στο stdin και στη συνέχεια να ελέγξε αν ήρθε η εντολή `TERMINATE`. Αυτή η σειρά ενεργειών δεν είναι επιθυμητή και για αυτό χρησιμοποιήθηκε η επικοινωνία μέσω pipe και της συνάρτησης `poll( )`. Επομένως η συνολική διαδρομή στο πρωτόκολλο `TERMINATE` είναι:

1. λήψη μηνύματος `TERMINATE` από terminal
2. αντιγραφή στο buffer σαν να ήταν ένα απλό μήνυμα και `post( )` τα write_threads
3. το κάθε write thread βλέπει την εντολή `TERMINATE` και μέσω του `pipe[1]` στέλνει στο read thread το -1 και τερματίζει
4. το read thread παίρνει είσοδο από `pipe[0]` βλέπει το -1 και τερματίζει και αυτό

5. η τελευταία διεργασία του κάθε διαλόγου τον θέτει ανενεργό
6. η τελευταία διεργασία όλης της εφαρμογής καταστρέφει τις δομές συγχρονισμού και τις αποδεσμεύει

4 Παραδείγματα

Ακολουθούν κάποια απλά παραδείγματα χρήσης:

```

sdi2200245@linux04:~/os1$ make run
./proc 2
Message received: GEIAA TI KANEIS?
kala esu ?
Message received: kala. xristougenna ti 8a kaneis?
edw xalara
Message received: aa ki egw na kanonisoume etsi
nai ennoeitai
epeidh exw douleia 8a milhsoume argotera
Message received: egine ante ta leme
Message received: TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ make run
./proc 2
GEIAA TI KANEIS?
Message received: kala esu ?
kala. xristougenna ti 8a kaneis?
Message received: edw xalara
aa ki egw na kanonisoume etsi
Message received: nai ennoeitai
Message received: epeidh exw douleia 8a milhsoume argotera
egine ante ta leme
TERMINATE
sdi2200245@linux04:~/os1$

```

Figure 1: Παράδειγμα ενός διαλόγου με 2 άτομα

```

sdi2200245@linux04:~/os1$ make run
./proc 2
GEIA POTE MPOREITE NA SUNANTHOUNE GIA THN ERGASIA?
Message received: EGN TRITH PEMPTH DE MPORW DOULEW
Message received: MPOREITE TETARTH
Message received: NAI
NAI
EGINE ANTE TA LEME
Message received: CIAOSSS
Message received: TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ make run
./proc 2
Message received: GEIA POTE MPOREITE NA SUNANTHOUNE GIA THN ERGASIA?
Message received: MPOREITE TETARTH
NAI
Message received: NAI
Message received: EGINE ANTE TA LEME
CIAOSSS
TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ make run
./proc 2
Message received: GEIA POTE MPOREITE NA SUNANTHOUNE GIA THN ERGASIA?
EGW TRITH PEMPTH DE MPORW DOULEW
MPOREITE TETARTH
Message received: NAI
Message received: NAI
Message received: EGINE ANTE TA LEME
Message received: CIAOSSS
Message received: TERMINATE
sdi2200245@linux04:~/os1$

```

Figure 2: Παράδειγμα ενός διαλόγου με 3 άτομα

```

sdi2200245@linux04:~/os1$ make run
./proc 2
Message received: GEIA TI KANEIS?
kala ESU
trexw me ergasies
Message received: aa mia apo ta idia
Message received: eee upomoh telelwoume siga siga
auto skleftomai sxedon telelwame
TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ make run
./proc 2
GEIA TI KANEIS?
Message received: kala ESU
Message received: trexw me ergasies
aa mia apo ta idia
eee upomoh telelwoume siga siga
Message received: auto skleftomai sxedon telelwame
Message received: TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ ./proc 1
Message received: GEIA
pame kana takidaki ta xristougenna?
kai de pame gia pou les?
Message received: de kserw, pame gia kafe avrio na to doume?
egine ti wra
Message received: ee kata tis 10
okk sto gnwsto?
Message received: nai egine ta leme
see u
TERMINATE
sdi2200245@linux04:~/os1$

sdi2200245@linux04:~/os1$ ./proc 1
GEIA
Message received: pame kana takidaki ta xristougenna?
Message received: kai de pame gia pou les?
de kserw, pame gia kafe avrio na to doume?
Message received: egine ti wra
ee kata tis 10
Message received: okk sto gnwsto?
nai egine ta leme
Message received: see u
Message received: TERMINATE
sdi2200245@linux04:~/os1$

```

Figure 3: Παράδειγμα δυο διαλόγων με 2 άτομα